BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 9

Roll no: 51 PRN No: 12411033 Name of Student: Manjiri Kulkarni

Subject Name: Deep Learning

Problem Statement:

Perform image classification using a pre-trained Vision Transformer (ViT) model and compareits performance with a CNN model.

Algorithm:

1. Start the program.

2. Import the required libraries, including PyTorch, Torchvision, Transformers, Matplotlib, and Pandas.

3. Load the CIFAR-10 dataset and preprocess the images by resizing them to \(224 \times 224\) pixels and normalizing them.

4. Select a subset of 5,000 training images and 2,000 testing images.

5. Load the pre-trained ResNet18 CNN model and replace its final classification layer with a 10-class output layer.

6. Load the pre-trained Vision Transformer (ViT) model and replace its classification head with a 10-class output layer.

7. Freeze the feature extraction layers of both models and configure the Adam optimizer and cross-entropy loss function.

8. Train the classification heads of both models for 2 epochs using the training dataset.

9. Evaluate both models on the test dataset and calculate their classification accuracy and evaluation time.

10. Generate a bar graph comparing the accuracy of the CNN and ViT models.

11. Generate a bar graph comparing the evaluation time of both models.

12. Display the performance comparison table and visualize sample images with their actual and predicted class labels.

13. Compare the performance of both models and display the results.

14. Stop the program.

Code:

```
!pip install -q torch torchvision transformers matplotlib pandas
import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as transforms
from torchvision.datasets import CIFAR10
from torch.utils.data import DataLoader, Subset
from transformers import ViTForImageClassification
import matplotlib.pyplot as plt
import pandas as pd
import time
# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
# Define classes
classes = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]
# Preprocess images
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
# Load CIFAR-10 dataset
train_dataset = CIFAR10(
    root="./data", train=True, download=True, transform=transform
```

```
)
test_dataset = CIFAR10(
    root="./data", train=False, download=True, transform=transform
)
# Select subsets
train_subset = Subset(train_dataset, range(5000))
test_subset = Subset(test_dataset, range(2000))
train_loader = DataLoader(
    train_subset, batch_size=32, shuffle=True
)
test_loader = DataLoader(
    test_subset, batch_size=32, shuffle=False
)
print("Training images:", len(train_subset))
print("Testing images:", len(test_subset))
# Visualize sample images
def show_image(img):
    mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
    std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)
    img = img.cpu() * std + mean
    return img.permute(1, 2, 0).clamp(0, 1)
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    img, label = test_dataset[i]
    ax.imshow(show_image(img))
    ax.set_title(classes[label])
    ax.axis("off")
plt.suptitle("Sample Images from CIFAR-10")
plt.tight_layout()
plt.show()
# Load pre-trained CNN
cnn = models.resnet18(
    weights=models.ResNet18_Weights.DEFAULT
)
cnn.fc = nn.Linear(cnn.fc.in_features, 10)
cnn = cnn.to(device)
# Load pre-trained Vision Transformer
vit = ViTForImageClassification.from_pretrained(
    "google/vit-base-patch16-224",
    num_labels=10,
    ignore_mismatched_sizes=True
)
vit = vit.to(device)
# Freeze feature extraction layers
for param in cnn.parameters():
    param.requires_grad = False
for param in cnn.fc.parameters():
    param.requires_grad = True
for param in vit.parameters():
    param.requires_grad = False
for param in vit.classifier.parameters():
```

```
    param.requires_grad = True
# Define loss and optimizers
criterion = nn.CrossEntropyLoss()
cnn_optimizer = torch.optim.Adam(
    cnn.fc.parameters(), lr=0.001
)
vit_optimizer = torch.optim.Adam(
    vit.classifier.parameters(), lr=0.001
)
# Train model for 2 epochs
def train_model(model, optimizer, is_vit, epochs=2):
    model.train()
    for epoch in range(epochs):
        total_loss = 0
        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()
            if is_vit:
                outputs = model(pixel_values=images).logits
            else:
                outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        print(
            f"Epoch {epoch + 1}/{epochs}, "
            f"Loss: {total_loss / len(train_loader):.4f}"
        )
    return model
print("\nTraining CNN...")
cnn = train_model(cnn, cnn_optimizer, False, epochs=2)
print("\nTraining ViT...")
vit = train_model(vit, vit_optimizer, True, epochs=2)
# Evaluate model accuracy and time
def evaluate_model(model, is_vit):
    model.eval()
    correct = 0
    total = 0
    if device.type == "cuda":
        torch.cuda.synchronize()
    start = time.time()
    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)
            labels = labels.to(device)
            if is_vit:
                outputs = model(pixel_values=images).logits
            else:
                outputs = model(images)
```

```
            predictions = outputs.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)
    if device.type == "cuda":
        torch.cuda.synchronize()
    elapsed = time.time() - start
    accuracy = 100 * correct / total
    return accuracy, elapsed
cnn_accuracy, cnn_time = evaluate_model(cnn, False)
vit_accuracy, vit_time = evaluate_model(vit, True)
print("\nCNN Accuracy:", round(cnn_accuracy, 2), "%")
print("CNN Evaluation Time:", round(cnn_time, 2), "seconds")
print("ViT Accuracy:", round(vit_accuracy, 2), "%")
print("ViT Evaluation Time:", round(vit_time, 2), "seconds")
# Plot accuracy comparison
model_names = ["CNN (ResNet18)", "Vision Transformer (ViT)"]
accuracies = [cnn_accuracy, vit_accuracy]
plt.figure(figsize=(8, 5))
bars = plt.bar(model_names, accuracies)
plt.title("CNN vs ViT - Accuracy Comparison")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 100)
for bar, value in zip(bars, accuracies):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 1,
        f"{value:.2f}%",
        ha="center"
    )
# Display results table
results = pd.DataFrame({
    "Model": model_names,
    "Accuracy (%)": accuracies,
    "Evaluation Time (seconds)": times
})
print("\nFinal Performance Comparison")
print(results.to_string(index=False))
# Display sample predictions
cnn.eval()
vit.eval()
images, labels = next(iter(test_loader))
images = images.to(device)
with torch.no_grad():
    cnn_predictions = cnn(images).argmax(dim=1)
    vit_predictions = vit(pixel_values=images).logits.argmax(dim=1)
fig, axes = plt.subplots(2, 5, figsize=(15, 7))
for i, ax in enumerate(axes.flat):
    img = show_image(images[i])
    actual = classes[labels[i].item()]
    cnn_pred = classes[cnn_predictions[i].item()]
    vit_pred = classes[vit_predictions[i].item()]
```

```
    ax.imshow(img)
    ax.set_title(
        f"Actual: {actual}\nCNN: {cnn_pred}\nViT: {vit_pred}",
        fontsize=8
    )
    ax.axis("off")
plt.suptitle("Image Classification: CNN vs ViT")
plt.tight_layout()
plt.show()
# Final comparison
print("\nConclusion:")
if cnn_accuracy > vit_accuracy:
    print("CNN achieved higher accuracy on the test subset.")
elif vit_accuracy > cnn_accuracy:
    print("ViT achieved higher accuracy on the test subset.")
else:
    print("Both models achieved the same accuracy.")
if cnn_time < vit_time:
    print("CNN had a shorter evaluation time.")
elif vit_time < cnn_time:
    print("ViT had a shorter evaluation time.")
else:
    print("Both models had equal evaluation times.")
print("Experiment completed successfully.")
```

Output:

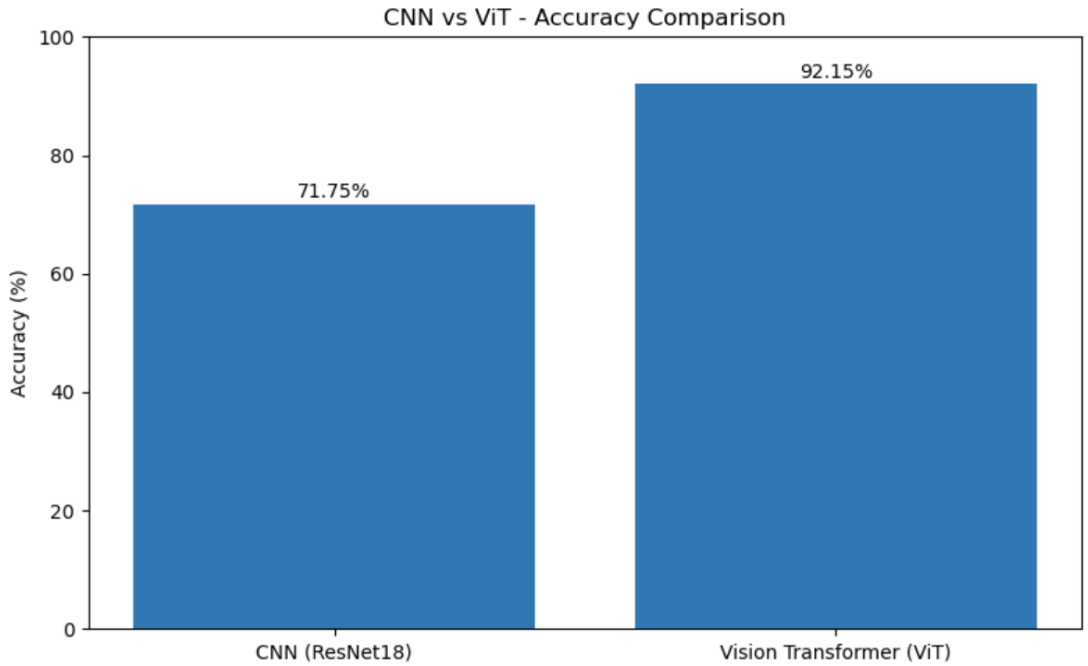


Image Classification Predictions: CNN vs ViT

